IBM Machine Learning Course Review

Module 1: Introduction to AI, ML, and DL

Artificial Intelligence (AI)

AI is a branch of computer science focused on simulating intelligent behavior in computers, enabling machines to mimic cognitive functions like learning and problem-solving.

Machine Learning (ML)

ML is a subfield of AI where algorithms learn patterns from data without explicit programming.

Deep Learning (DL)

DL is a subset of ML that utilizes multi-layered neural networks to learn from vast amounts of data. It combines feature recognition and classification algorithms.

History of AI

- **1950s:** Alan Turing proposed the Turing Test; AI accepted as a field at the Dartmouth Conference (1956); Frank Rosenblatt invented the perceptron (1957); Arthur Samuel developed a checkers program using ML (1959).
- **AI Winters:** Periods of reduced funding and interest due to unmet expectations.
    - **First AI Winter (Late 1960s - 1970s):** Fueled by the ALPAC report on machine translation (1966), Minsky's book on perceptron limitations (1969), and the Lighthill report (1973).
    - **Second AI Winter (Late 1980s - 1990s):** Expert systems became less viable as they moved to PCs; neural networks struggled with scalability.
- **1980s AI Boom:** Expert systems gained traction; the "Backpropagation" algorithm (1986) revived interest in neural networks.
- **1990s - 2000s:** ML solutions achieved success in various domains (speech recognition, medical diagnosis); Deep Blue beat Garry Kasparov (1997); Google Search utilized AI.
- **Modern AI (2012-Present):** Driven by larger datasets, faster computers, and advancements in neural networks (rebranded as Deep Learning after 2006). Key milestones include breakthroughs in ImageNet competitions (2012), natural language understanding (2013-2014), and advanced applications like self-driving cars and AI debate systems.

Modern AI Differences

- **Bigger Datasets:** Availability of massive datasets.
- **Faster Computers:** Increased computational power.
- **Neural Networks:** Advanced architectures leading to cutting-edge results.

Machine Learning Workflow

1. **Problem Statement:** Clearly define the problem to be solved.
2. **Data Collection:** Gather relevant data.
3. **Data Exploration & Processing:** Clean, transform, and prepare the data.
4. **Modeling:** Select and train ML algorithms.
5. **Validation:** Evaluate model performance.
6. **Decision Making & Deployment:** Interpret results and deploy the model.

Vocabulary

- **Target:** The category or variable to be predicted.
- **Features:** Explanatory variables used for prediction.
- **Example:** A single data point or observation (a row in a dataset).
- **Label:** The target value for a single data point.

Module 2: Retrieving and Cleaning Data

Data Retrieval

Data can be retrieved from various sources:

- **Flat Files:**
  - **CSV (Comma-Separated Values):** Rows separated by commas.
    - `pd.read_csv(filepath)`
    - Arguments: `sep` (for different delimiters like `'\t'` for TSV), `delim_whitespace=True`, `header=None` (if no header row), `names=['col1', 'col2']` (to specify column names).
  - **JSON (JavaScript Object Notation):** Standard for data storage across platforms, similar to Python dictionaries.
    - `pd.read_json(filepath)`
    - `data.to_json('outputfile.json')` (to write a DataFrame to JSON).

- **SQL Databases:** Relational databases with fixed schemas.
    - Examples: Microsoft SQL Server, PostgreSQL, AWS Redshift, Oracle DB, Db2.
    - Reading SQL data:

```
import sqlite3 as sq3
import pandas as pd

path = 'data/classic_rock.db'
con = sq3.connect(path)
query = "SELECT * FROM roch_songs;"
data = pd.read_sql(query, con)
```

- **NoSQL Databases:** Non-relational databases with varying structures, often storing data in JSON format.
    - Examples: MongoDB, Cassandra, Redis.
    - Reading NoSQL (MongoDB) data:

```
from pymongo import MongoClient

con = MongoClient()
db = con.database_name # Choose a database
cursor = db.collection_name.find(query) # Execute a query
df = pd.DataFrame(list(cursor)) # Convert cursor to DataFrame
```

- **APIs and Cloud Data Access:** Accessing data directly from web services or cloud storage.
    - Example: Reading data from a URL.

```
data_url = 'http://archive.ics.uci.edu/ml/machine-learning-
databases/autos/imports-85.data'
df = pd.read_csv(data_url, header=None)
```

Data Cleaning

Data cleaning is crucial for reliable ML outcomes. Messy data can lead to inaccurate models.

Importance of Data Cleaning

- Ensures data accuracy and reliability for ML algorithms.
- Improves model performance and decision-making.

Common Data Problems

- **Lack of data:** Insufficient data for analysis.
- **Too much data:** Overwhelming volume that needs processing.
- **Bad data:** Inaccurate, incomplete, or inconsistent data.

How Data Can Be Messy

- **Duplicate or unnecessary data:** Redundant entries.
- **Inconsistent text and typos:** Variations in spelling or formatting.
- **Missing data:** Gaps in the dataset.
- **Outliers:** Extreme values that deviate significantly from the rest.
- **Data source issues:** Inconsistencies from multiple systems or database types.

Dealing with Duplicate or Unnecessary Data

- Identify duplicate values and investigate their origin.
- Filter data as needed, but be cautious not to remove potentially useful features.

Working with Missing Values

- **Remove:** Delete rows or columns with missing data.
- **Impute:** Replace missing values with substituted values (e.g., mean, median, mode).
- **Mask:** Create a new category specifically for missing values.

Outliers

An observation distant from others. They can significantly impact models if not handled. Some outliers can be informative.

How to Find Outliers

- **Plots:** Histograms, Density Plots, Box Plots.

```
import seaborn as sns
sns.distplot(data, bins=20) # Histogram
sns.boxplot(data) # Boxplot
```

- **Statistics:** Using measures like the Interquartile Range (IQR).

```
import numpy as np

q25, q75 = np.percentile(data, [25, 75])
iqr = q75 - q25
min_limit = q25 - 1.5 * iqr
max_limit = q75 + 1.5 * iqr

# Example of finding outliers in a specific column 'Unemployment'
outliers = [x for x in data['Unemployment'] if x > max_limit or x < min_limit]
```

- **Residuals:** Differences between actual and predicted values.

Policies for Dealing with Outliers

- **Remove:** Delete outlier data points.
- **Impute:** Replace outliers with the mean or median.
- **Transform:** Apply mathematical transformations to the variable (e.g., log transformation).
- **Predict:** Use models to predict what the outlier value should be based on similar observations or regression.

Module 3: Exploratory Data Analysis (EDA) and Feature Engineering

Exploratory Data Analysis (EDA)

EDA is an approach to analyzing datasets to summarize their main characteristics, often using visual methods.

Why EDA is Useful

- Provides an initial understanding of the data.
- Helps determine if further cleaning or more data is needed.
- Identifies patterns, trends, and relationships in the data.

Techniques for EDA

- **Summary Statistics:** Mean, median, min, max, correlations.
- **Visualizations:** Histograms, scatter plots, box plots.

Tools for EDA

- **Data Wrangling:** Pandas.
- **Visualization:** Matplotlib, Seaborn.

Sampling from DataFrames

- `data.sample(n=5, replace=False)`: Samples 5 rows without replacement.
- Reasons for sampling: simplify computation for large datasets, train models, over/under-sample for imbalanced outcomes.

EDA with Visualization Libraries

- **Matplotlib:** Primary library for creating plots, offering flexibility.
  - Basic plotting: `plt.plot(x_data, y_data, ls='', marker='o')`
  - Customizing plots: Using `fig, ax = plt.subplots()` for object-oriented control over axes, labels, and titles.
- **Pandas:** Provides convenient wrappers around Matplotlib for easier plotting, especially with DataFrames.
  - Plotting grouped data: `data.groupby('species').mean().plot(...)`
- **Seaborn:** Built on Matplotlib, simplifies creating aesthetically pleasing and statistically informative plots.
  - **Scatter Plots:** Differentiate data points by color and add legends.
  - **Histograms:** Visualize the distribution of a single variable.
  - **Pair Plots (`sns.pairplot`):** Visualize pairwise relationships between features, often with hue for categorical variables.
  - **Hexbin Plots (`sns.jointplot(kind='hex')`):** Shows density of points in hexagonal bins.
  - **Facet Grids (`sns.FacetGrid`):** Create plots across subsets of data based on categorical variables.

Feature Engineering and Variable Transformation

Models often assume specific data properties (e.g., linear relationships in linear regression). Transformations can help meet these assumptions.

Transformation of Data Distributions

- **Log Transformations:** Useful for linear regression when dealing with skewed data. Can be applied using `numpy.log` or `numpy.log1p`.

- **Polynomial Features:** Create higher-order terms of existing features to capture non-linear relationships using linear models.

```
from sklearn.preprocessing import PolynomialFeatures

polyFeat = PolynomialFeatures(degree=2) # Create instance for degree 2
X_poly = polyFeat.fit_transform(X_data) # Fit and transform data
```

Feature Encoding

Converting non-numeric features into numeric ones.

- **Types of Categorical Features:**
  - **Nominal:** Unordered categories (e.g., colors: Red, Blue).
  - **Ordinal:** Ordered categories (e.g., size: High, Medium, Low).
- **Encoding Approaches:**
  - **Binary Encoding:** Converts variables with two values to 0 or 1.
  - **One-Hot Encoding:** Creates new binary (0/1) columns for each category in a nominal variable.
  - **Ordinal Encoding:** Converts ordered categories to numerical values (e.g., 0, 1, 2).

Feature Scaling

Adjusting the scale of numeric data so features are comparable.

- **Scaling Approaches:**
  - **Standard Scaling:** Converts features to a standard normal distribution (mean=0, std dev=1).
    - `StandardScaler()`
  - **Min-Max Scaling:** Scales features to a specific range (usually [0, 1]). Sensitive to outliers.
    - `MinMaxScaler()`
  - **Robust Scaling:** Scales features using the interquartile range (IQR), making it less sensitive to outliers than Min-Max scaling.
    - `RobustScaler()`

Common Variable Transformations Summary

Feature Type	Transformation	Example Methods
Continuous	Scaling	Standard, Min-Max, Robust Scaling
Nominal (Unordered Cat.)	Encoding	Binary, One-Hot Encoding
Ordinal (Ordered Cat.)	Encoding	Ordinal Encoding

Module 4: Inferential Statistics and Hypothesis Testing

Introduction to Inference

- **Estimation:** Applying an algorithm to derive a value from data (e.g., calculating an average).
- **Inference:** Quantifying the accuracy or uncertainty of an estimate (e.g., calculating the standard error of an average).
- Both ML and statistical inference aim to learn about the underlying data-generating process (DPG).

Parametric vs. Non-Parametric Statistics

- **Parametric Models:** Assume data comes from a specific distribution with a finite number of parameters (e.g., Normal distribution).
    - **Maximum Likelihood Estimation (MLE):** A common method for estimating parameters in parametric models.
    - **Common Distributions:** Uniform, Gaussian/Normal, Log Normal, Exponential, Poisson.
- **Non-Parametric Statistics:** Make fewer assumptions about the data distribution (distribution-free inference).
    - Example: Creating a cumulative distribution function (CDF) using a histogram without assuming specific parameters.

Frequentist vs. Bayesian Statistics

- **Frequentist:** Concerned with the long-run frequency of events in repeated trials. Derives probabilistic properties of procedures and applies them to observed data.
- **Bayesian:** Describes parameters using probability distributions. Starts with a **prior distribution** (belief before data) and updates it using data to get a **posterior distribution**.

Hypothesis Testing

A formal procedure to make decisions about population parameters based on sample data.

- **Hypotheses:**
  - **Null Hypothesis (H_0):** A statement of no effect or no difference.
  - **Alternative Hypothesis (H_1 or H_A):** A statement that contradicts the null hypothesis.
- **Decision Rules:** Based on a test statistic, decide whether to reject H_0 in favor of H_1.
- **Errors:**
  - **Type I Error:** Rejecting H_0 when it is actually true (False Positive).
  - **Type II Error:** Failing to reject H_0 when it is false (False Negative).
- **Terminology:**
  - **Test Statistic:** A value calculated from sample data used to decide between hypotheses.
  - **Rejection Region:** The set of test statistic values that lead to rejecting H_0.
  - **Acceptance Region:** The set of test statistic values that lead to accepting H_0.
  - **Null Distribution:** The distribution of the test statistic when the null hypothesis is true.
- **Significance Level (α):** The probability of making a Type I error, chosen *before* testing. A common value is 0.05.
- **p-value:** The smallest significance level at which the null hypothesis would be rejected. If p-value < α, reject H_0.
- **Confidence Interval:** The range of values for the statistic for which we would accept the null hypothesis.

Example: Coin Tossing

- **Scenario:** Observe 3 heads in 10 flips.
- **Hypotheses:**
  - H_0: The coin is fair ($P(H) = 0.5$).
  - H_1: The coin is unfair ($P(H) \neq 0.5$).
- **Testing:**
  - The number of heads in 10 flips follows a binomial distribution: $X \sim \text{Binomial}(n = 10, p = 0.5)$.
  - Calculate the probability of observing 3 or fewer heads (or 7 or more heads, depending on the defined H_1): $P(X \leq 3) \approx 0.171$ (or 17.1%).
  - If we set a significance level $\alpha = 0.05$, since $0.171 > 0.05$, we **do not reject** H_0. The observed result is not statistically significant enough to conclude the coin is unfair.

Marketing Intervention Example

- H_0: The marketing campaign does not impact purchasing.

- H_1: The marketing campaign has an impact on purchasing.

Website Layout Example

- H_0: The website layout change has no impact on traffic.
- H_1: The website layout change has an impact on traffic.

Product Quality Example

- H_0: Product size is not significantly different from the standard size S.
- H_1: Product size is significantly different from the standard size S.

F-Statistic

Used in regression models. H_0 typically states that all regression coefficients (β) are zero. Reject H_0 if the p-value is small.

Power and Sample Size

- **Power:** The probability of correctly rejecting H_0 when it is false.
- Increasing significance tests increases the chance of Type I errors.
- **Bonferroni Correction:** A method to control the overall Type I error rate when performing multiple comparisons, by adjusting the significance level for each test. It reduces power.

Module 2 (Review): Retrieving and Cleaning Data

Data Retrieval Summary

Data can be retrieved from SQL databases, NoSQL databases, APIs, and cloud sources. CSV and TSV are common flat file formats. SQL databases are relational, while NoSQL databases are generally non-relational and often use JSON.

Data Cleaning Summary

Data cleaning is vital because messy data leads to unreliable outcomes. Common issues include:

- Duplicate or unnecessary data.
- Inconsistent data and typos.
- Missing data.
- Outliers.
- Data source issues.

Handling Missing Data:

- Remove rows/columns.
- Impute values (e.g., mean, median).
- Mask data by creating a category.

Identifying and Handling Outliers: Methods include plots, statistics (IQR), and residuals. Policies include removal, imputation, transformation, or prediction.

Module 3 (Review): Exploratory Data Analysis and Feature Engineering

EDA Summary

EDA helps understand data characteristics, identify patterns, and determine if more cleaning or data is needed, often using visualizations and summary statistics. Libraries like Pandas, Matplotlib, and Seaborn are key tools.

Feature Engineering Summary

Feature engineering involves transforming variables to improve model performance.

- **Transformations:** Log transformations, polynomial features can address data distribution issues and non-linear relationships.
- **Encoding:** Converts categorical variables (nominal and ordinal) into numerical formats using methods like one-hot encoding or ordinal encoding.
- **Scaling:** Adjusts the scale of numerical features using methods like Standard Scaling, Min-Max Scaling, or Robust Scaling to ensure comparability.